第4章　派生类与继承

第四章 派生类与继承

1. 派生类与继承的基本概念
2. 派生类的定义
3. 派生类的构造函数与析构函数
4. 多继承

1. 派生类与继承的基本概念　为什么要使用继承？

✓在自然界中，继承这个概念是非常普遍的。小猫仔继承了猫爸猫妈的特性，所以长得是猫鼻子猫眼，我们不会把它错认为是小狗。继承就是这样，会将一些本质的特性遗传给子代，使子代在很大程度上具有与父代相同的性质。当然，子代同时还具有父代没有的特性。

✓从上面的例子可以看出，一旦指定了某种事物父代的本质特征，那么它的子代将会自动具有那些性质。这就是一种朴素的**可重用**的概念。而且子代可以拥有父代没有的特性，这是**可扩充**的概念。

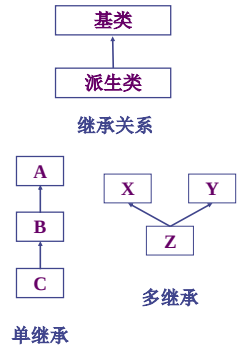
✓**继承**：在一个已经存在的类的基础上建立另一个新的类。建立的新的类不仅拥有新定义的成员（属性和方法），而且还同时拥有旧的成员（属性和方法）——**实现代码重用**。

1. 派生类与继承的基本概念　基类与派生类

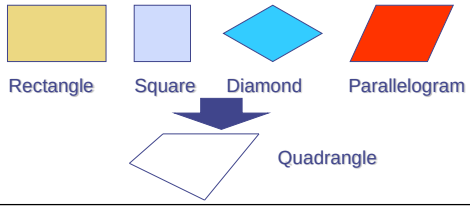
✓我们称已存在的用来派生新的类型的类为**基类**，又称为**父类**。由已存在的类派生出的新类称为**派生类**，又称为**子类**。

✓在C++语言中，一个派生类可以从一个父类派生，从一个父类派生的继承称**单继承**，也可以从多个父类派生，从多个父类派生的继承称**多继承**。

✓派生类继承了基类的数据成员和成员函数，并增加了自己新的成员。



1. 派生类与继承的基本概念　现实世界的例子



- 四边形是一个基本的概念，矩形、正方形、菱形和平行四边形都是派生的概念，因为它们都属于四边形，它们继承了四边形的所有性质，同时又扩充出各自的特性。

第四章 派生类与继承

1. 派生类与继承的基本概念
2. 派生类的定义
3. 派生类的构造函数与析构函数
4. 多继承

2. 派生类的定义

基类与派生类

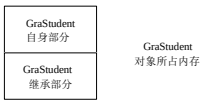
✓**单继承：**

class 派生类名：继承方式 基类名
{
派生类新定义成员
};

✓**三种继承方式：**

public： 公有继承
private：私有继承（默认的方式）
protected：保护继承

✓**系统的默认值是私有继承**




```
class Student
  { //…… };

class GraStudent :public Student
  { //…… };

void fn(Student & s)
  { //…… }

int main()
  { GraStudent gs;
    fn(gs);}
```

在构造GraStudent对象时，系统首先要构造Student对象，然后再构造自身，从而拼接出一个完整的GraStudent对象。

2. 派生类的定义

公有继承

以公有继承的派生类对基类成员访问权限：

✓基类公有成员相当于派生类的公有成员，即派生类可以象访问自身公有成员一样访问从基类继承的公有成员。

✓基类保护成员相当于派生类的保护成员，即派生类可以象访问自身的保护成员一样，访问基类的保护成员。

✓对于基类的私有成员，派生类无法直接访问。

➢**public**是通常的继承形式。

```
class Stu
  { //…… };

class GraStu :public Stu
  { //…… };

void fn(Stu & s)
  { //…… }

int main()
  { GraStu gs;
    fn(gs);}
```


2. 派生类的定义

保护继承

以保护继承的派生类对基类成员访问权限：

✓基类的公有成员和保护成员都相当于派生类的保护成员，派生类可以通过自身的成员函数或其子类的成员函数访问它们。

✓对于基类的私有成员，无论派生类**内部成员**或派生类**使用者**都无法直接访问。

派生类成员函数

派生类对象

```
class Stu
  { //…… };

class GraStu :protected Stu
  { //…… };

void fn(Stu & s)
  { //…… }

int main()
  { GraStu gs;
    fn(gs);}
```


2. 派生类的定义

私有继承

以私有继承的派生类对基类成员访问权限：

✓基类公有成员和保护成员都相当于派生类的私有成员，派生类只能通过自身的函数成员访问它们；

✓对于基类的私有成员，无论派生类内部成员或派生类使用者都无法直接访问。

➢**private**继承形式通常不用。

```
class Stu
  { //…… };

class GraStu :private Stu
  { //…… };

void fn(Stu & s)
  { //…… }

int main()
  { GraStu gs;
    fn(gs);}
```


2. 派生类的定义

三种继承方式总结

派生类对基类成员的访问能力

继承方式 / 基类成员	公有继承 public	保护继承 protected	私有继承 private
公有成员 public	public	protected	private
保护成员 protected	protected	protected	Private
私有成员 private	不能继承	不能继承	不能继承

不同继承方式的影响主要体现在：

✓派生类**成员函数**对基类成员的访问权限

✓通过派生类**对象**对基类成员的访问权限

2. 派生类的定义

例：继承方式

```
#include <iostream.h>
class A
{ public:
    A()  {i1 = 10; j1 = 11;}
    void f1();
  protected:   int  j1;
  private:     int  i1;
};

class B:public A
{ public:
    B() {i2 = 20; j2 = 21;}
    void  f2();
  protected:  int  j2;
  private:    int  i2;
};
```

```
class C:public B
{ public:
    C() {i3 = 30; j3 = 31;}
    void  f3();
  protected:   int j3;
  private:     int i3;
};

void main()
{  A a;   B b;   C c; ….}
```

1. 派生类B中成员函数f2()能否访问基类A中的成员f1()，i1和j1?

2. 派生类的定义

例：继承方式

```
#include <iostream.h>
class A
{ public:
    A()  {i1 = 10; j1 = 11;}
    void f1();
  protected:   int j1;
  private:     int i1;
};

class B:public A
{ public:
    B() {i2 = 20; j2 = 21;}
    void  f2();
  protected:   int j2;
  private:     int i2;
};
```

```
class C:public B
{ public:
    C() {i3 = 30; j3 = 31;}
    void  f3();
  protected:   int j3;
  private:     int i3;
};

void main()
{  A a;   B b;   C c; ….}
```

2. 派生类B的对象b能否访问基类A中的成员f1()，i1和j1?

13

2. 派生类的定义

例：继承方式

```
#include <iostream.h>
class A
{ public:
    A()  {i1 = 10; j1 = 11;}
    void f1();
  protected:   int j1;
  private:     int i1;
};

class B:public A
{ public:
    B() {i2 = 20; j2 = 21;}
    void  f2();
  protected:   int j2;
  private:     int i2;
};
```

```
class C:public B
{ public:
    C() {i3 = 30; j3 = 31;}
    void  f3();
  protected:   int j3;
  private:     int i3;
};

void main()
{  A a;   B b;   C c; ….}
```

3. 派生类C中成员函数f3()能否访问直接基类B中的成员f2()，i2和j2和间接基类A中的成员f1()，i1和j1?

14

2. 派生类的定义

例：继承方式

```
#include <iostream.h>
class A
{ public:
    A()  {i1 = 10; j1 = 11;}
    void f1();
  protected:   int j1;
  private:     int i1;
};

class B:public A
{ public:
    B() {i2 = 20; j2 = 21;}
    void  f2();
  protected:   int j2;
  private:     int i2;
};
```

```
class C:public B
{ public:
    C() {i3 = 30; j3 = 31;}
    void  f3();
  protected:   int j3;
  private:     int i3;
};

void main()
{  A a;   B b;   C c; ….}
```

4、派生类C的对象c能否访问直接基类B中的成员f2(),i2和j2？能否访问间接基类A中的成员f1()，i1和j1?

15

2. 派生类的定义

问题与解答

1. 派生类B中成员函数f2()能否访问基类A中的成员f1()，i1和j1?（可以访问f1()和j1，不可访问i1）

2. 派生类B的对象b能否访问基类A中的成员f1()，i1和j1?（可以访问f1()，不可访问i1和j1）

3. 派生类C中成员函数f3()能否访问直接基类B中的成员f2()，i2和j2？能否访问间接基类A中的成员f1()，i1和j1?（可访问f2()、j2、f1()和j1，不可访问i2及i1）

4. 派生类C的对象c能否访问直接基类B中的成员f2(),i2和j2？能否访问间接基类A中的成员f1()，i1和j1?（可访问直接基类中的f2()和间接基类中的f1()，其它不可访问）

结论：不论哪种继承，派生类成员函数可访问基类中public和protected成员；仅公有继承中派生类对象可访问基类中的public成员。事实上，派生类对象也仅能访问自己的public成员！！

16

2. 派生类的定义 派生类与基类同名成员的访问方式

✓C++允许派生类可以重新定义基类的成员，此时称派生类的成员**覆盖**了基类的同名成员。

✓如果在派生类中，或通过派生类对象来使用基类的同名成员，则可以显式地使用**作用域限定符**，方式如下：

基类名::成员

```
void main( )
  {   Derived obj(5,6);
      obj.v2=8;  //错误
      obj.Base::v2=9;
  }
```

```
class Base
 { protected: int v1;
   public:  int v2;
     Base(int a=0,int b=0)
       {  v1=a;v2=b;}   };

class Derived : public Base
 { int v2;
   public: int v3;
     Derived(int a=0,int b=0)
       { v2=a;v3=b; }
     void func()
       { int sum1=v1+v2+v3;
         int sum2=v1+Base::v2+v3;}};
```

17

2. 派生类的定义 小结

- 当从现成的类派生出新类时，派生类将继承基类除私有成员以外的所有特性（成员），还可以有以下功能：
  - **增加新的成员**
  - **重新定义成员**
- 从编码的角度来看，派生类从基类中以较低的代价换来了较大的灵活性。一旦产生了可靠的基类，只需要调试派生类中所做的修改即可。派生类可以对继承的属性进行扩展、限制或改变。

18

2. 派生类的定义

对象赋值兼容规则

- 在公有派生方式下，派生类对象可以作为基类对象来使用：
  - 派生类的对象可以直接赋值给基类的对象

```
class Student
  { //…… };
class GraStudent :public Student
  { //…… };
void fn1(Student s)
  { //…… }
void fn2(Student & s)
  { //…… }
void fn3(Student * s)
  { //…… }

int main()
  { GraStudent gs;
    Student ss=gs;
  }
```


2. 派生类的定义

对象赋值兼容规则

- 在公有派生方式下，派生类对象可以作为基类对象来使用：
  - 派生类的对象可以直接赋值给基类的对象
  - 基类对象的引用可以引用一个派生类对象

```
class Student
  { //…… };
class GraStudent :public Student
  { //…… };
void fn1(Student s)
  { //…… }
void fn2(Student & s)
  { //…… }
void fn3(Student * s)
  { //…… }

int main()
  { GraStudent gs;
    Student & ss=gs;
  }
```


2. 派生类的定义

对象赋值兼容规则

- 在公有派生方式下，派生类对象可以作为基类对象来使用：
  - 派生类的对象可以直接赋值给基类的对象
  - 基类对象的引用可以引用一个派生类对象
  - 基类对象的指针可以指向一个派生类对象

```
class Student
  { //…… };
class GraStudent :public Student
  { //…… };
void fn1(Student s)
  { //…… }
void fn2(Student & s)
  { //…… }
void fn3(Student * s)
  { //…… }

int main()
  { GraStudent gs;
    Student *ss= &gs; }
```


2. 派生类的定义

对象赋值兼容规则

- 在公有派生方式下，派生类对象可以作为基类对象来使用：
  - 派生类的对象可以直接赋值给基类的对象
  - 基类对象的引用可以引用一个派生类对象
  - 基类对象的指针可以指向一个派生类对象
  - 如果函数的形参是基类对象或基类对象的引用或基类对象的指针，在调用时可以用派生类对象

```
class Student
  { //…… };
class GraStudent :public Student
  { //…… };
void fn1(Student s)
  { //…… }
void fn2(Student & s)
  { //…… }
void fn3(Student * s)
  { //…… }

int main()
  { GraStudent gs;
    fn1(gs);
    fn2(gs);
    fn3(&gs); }
```


2. 派生类的定义

对象赋值兼容规则

- 几点说明：
  - 以上赋值关系仅限公有继承
  - 只能使用从基类继承来的成员
  - 不可以将基类对象赋值给派生类对象

```
class Student
  { //…… };
class GraStudent :public Student
  { //…… };

int main()
  { GraStudent gs;
    Student ss;
    gs=ss; //  ×
  }
```

第四章 派生类与继承

1. 派生类与继承的基本概念
2. 派生类的定义
3. 派生类的构造函数与析构函数
4. 多继承

3. 派生类的构造函数与析构函数
派生类构造函数

➢构造函数和析构函数都不能够被继承。

➢派生类在构造时先要调用父类的构造函数，再是派生类中子对象(类成员)的构造函数，最后调用派生类的构造函数。

派生类名::派生类名（参数总表）:基类名（参数表1），…，基类名（参数表n），子对象名1（对象参数表1），…，子对象名m（对象参数表m）
{
派生类新增加成员的初始化；
}

如果基类使用的是缺省的构造函数或不带参数的构造函数，那么在初始化列表中可以省略“基类名（参数表1），…，基类名（参数表n）”这些项。

25

3. 派生类的构造函数与析构函数
派生类构造函数

```
class Advisor
{  int noOfMeeting;};

class Student
{public:
  Student(char* pName ="no name")
   { strncpy(name, pName, sizeof(name));
     ……;  }
  protected:
  char name[40];
  ……;};

class GStudent :public Student
{public:
    GStudent(char * pname,Advisor adv):Student(pname), advisor(adv)
     {qualifierGrade=0;}
  protected:
    Advisor advisor;
    int qualifierGrade;};
```

✓当基类只有有参构造函数时，派生类必须定义构造函数，以提供将参数传递给基类构造函数的途径。因此在有些情况下，派生类的构造函数的函数体可能为空，仅起到参数传递的作用。

✓构造顺序：

调用基类的构造函数
调用成员对象的构造函数
调用派生类构造函数。

构造 ↑ ↓ 析构

✓析构则严格按照与构造相反的次序。

✓成员对象初始化顺序依类中声明的顺序。

26

3. 派生类的构造函数与析构函数
派生类构造函数

派生类构造函数总结：

(1) 派生类的构造函数名与派生类名相同。

(2) 参数表需要列出初始化基类数据、新增内嵌对象数据及新增一般数据成员所需要的全部参数。

(3) 冒号之后，列出需要使用参数进行初始化的基类名和内嵌成员名及各自的参数表，各项之间用逗号分隔。

(4) 在定义派生类对象时构造函数的执行顺序是先祖先（基类，调用顺序按照它们继承时说明的顺序），再客人（对象成员，调用顺序按照它们在类中说明的顺序），后自己（派生类本身）。

27

3. 派生类的构造函数与析构函数
派生类析构函数

➢派生类在析构时，由于析构函数不能被继承，因此在执行派生类的析构函数时，父类的析构函数也将被调用，析构函数的调用顺序正好与构造函数相反。

➢析构过程相对简单，由于每个类最多只能有一个析构函数，不存在重载的问题，所以在派生类的析构过程中不必指出要调用父类哪个析构函数，只需按常规的格式编写派生类的析构函数。

派生类名::~派生类名（）
{
函数体
}

28

3. 派生类的构造函数与析构函数
派生类析构函数

派生类析构函数总结：

- 派生类与基类的析构函数没有什么联系，彼此独立，派生类或基类的析构函数只作各自类对象消亡前的善后工作。
- 在派生过程中，基类的析构函数不能继承，如果派生类需要析构函数的话，就要在派生类中重新定义。派生类析构函数的定义方法与没有继承关系的类中析构函数的定义方法完全相同，只要在函数体中负责把派生类新增的非对象成员的清理工作做好就够了，系统会自己调用基类及成员对象的析构函数来对基类及对象成员进行清理。
- 析构函数的执行顺序和构造函数正好严格相反：先自己（派生类本身），再客人（对象成员），后祖先（基类）。

29

3. 派生类的构造函数与析构函数
例：派生类构造与析构

```
#include <iostream.h>
class A
 { public: A() {cout <<"A Constructor1"<< endl;  }
           A(int i) {x1 = i;  cout <<"A Constructor2"<< endl;}
           void dispa() {cout <<"x1="<< x1 <<endl;}
           ~A(){ cout <<"A Destructor"<<endl;}
   protected:  int x1;            };

class B : public A
 { public: B() {cout <<"B Constructor1"<< endl;   }
            B(int i): A(i+10)
              {x2 = i;cout <<"B Constructor2"<< endl;}
           void dispb() {dispa();cout <<"x2="<< x2 <<endl;}
           ~B() { cout <<"B Destructor"<<endl;}
   protected:  int x2;            };

void main()
  {  B b(2);   b.dispb();
  }
```

输出：
A Constructor2
B Constructor2
X1=12
X2=2
B Destructor
A Destructor

30

3. 派生类的构造函数与析构函数

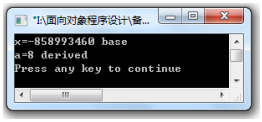
思考题

```
#include<iostream.h>
class base
  { int x;
    public:
      base(int i):x(i)
        {  cout<<"x="<<x<<" "
                <<"base\n"; }
    };

class derived:public base
   {  int a;
      public:
        derived(int i):a(i*4),base(a)
          {cout<<"a="<<a<<" "
                <<"derived\n"; }
};
```

```
int main()
{
  derived obj(2);
  return 0;
}
```

程序运行后的输出是？



第四章 派生类与继承

1. 派生类与继承的基本概念
2. 派生类的定义
3. 派生类的构造函数与析构函数
4. 多继承

4. 多继承

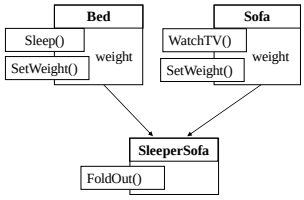
多继承派生类的声明

➢多继承——一个类由多个不同的直接基类派生，格式：

class 派生类名:继承方式1 父类名1,继承方式2 父类名2,…
{派生类新定义成员};

class SleeperSofa :public Bed, public Sofa
{ ……};

在实际编程中，几乎不可能需要编写从多个父类继承的子类，特别是在Visual C++环境下使用MFC类库进行编程时，这种情况几乎不会发生。



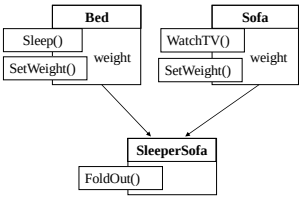
4. 多继承

多继承的二义性问题

多继承的关键是解决二义性问题：

1.访问不同直接基类的具有相同名字成员时可能出现二义性。

•**解决方案**：用类名对成员加以限定。但非最佳方案：如果相同的成员代表相同的属性和方法，我们根本不需要来自不同直接基类的多个拷贝——浪费内存资源！




```
SleeperSofa ssofa;
ssofa.weight; // 二义性
ssofa.SetWeight(); // 二义性
ssofa.Bed::weight; // ok
ssofa.Bed::SetWeight(); // ok
```


4. 多继承

多继承的二义性问题

```
#include<iostream.h>

class Bed{
public:
  Bed() :weight(0){}
  void Sleep(){ cout <<"Sleeping...\n"; }
  void SetWeight(int i){ weight =i; }
protected:
  int weight;
};

class Sofa{
public:
  Sofa() :weight(0){}
  void WatchTV(){ cout <<"Watching TV.\n"; }
  void SetWeight(int i){ weight =i; }
protected:
  int weight;
};

class SleeperSofa :public Bed, public Sofa{
public:
  SleeperSofa(){}
  void FoldOut(){ cout <<"Fold out the sofa.\n"; }
};

void main()
{
  SleeperSofa ss;
  ss.WatchTV();
  ss.FoldOut();
  ss.Sleep();
  ss.Sofa::SetWeight(20);
}
```

➢Bed类和Sofa类间没有任何继承与被继承关系，但二者都有同名同参数的成员函数SetWeight()。这时，如果SleeperSofa类从这两个类继承来，那么SleeperSofa类将有两个同名同参数成员函数，要在SleeperSofa类中访问SetWeight()就必须要用域运算符(::)标明调用是从哪个类继承来的，如果SleeperSofa类中也定义有同名同参数的成员函数，调用时不给出具体的父类名和域运算符，则是调用该类自己的成员函数。

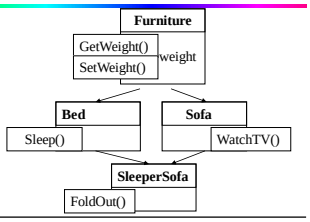
4. 多继承

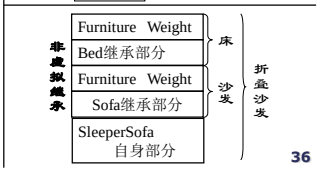
多继承的二义性问题

2.访问共同的间接基类的成员时可能出现二义性。

优化：增加间接基类：
Furniture
仍然没有解决二义性与内存资源浪费问题！

✓Sofa类和Bed类分别继承Furniture类，SleeperSofa继承Bed类和Sofa类，这时，SleeperSofa将有两个来自Furniture的成员，会产生二义。





4. 多继承

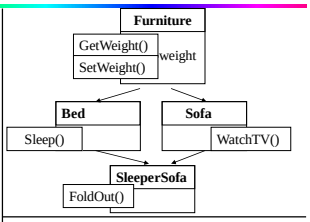
多继承的二义性问题

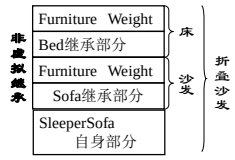
2.访问共同的间接基类的成员时可能出现二义性。

优化：增加间接基类：Furniture

仍然没有解决二义性与内存资源浪费问题！

```
SleeperSofa ssofa;
ssofa.weight; // 二义性
ssofa.SetWeight(); // 二义性
ssofa.Bed::weight; // ok
ssofa.Bed::SetWeight(); //ok
```





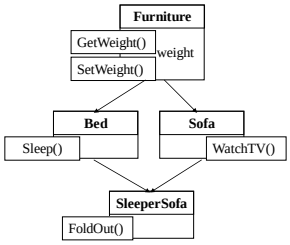
4. 多继承

虚继承（虚基类）

✓解决二义性问题的根本办法是引入虚继承的概念

✓**虚继承**：在建立继承关系时，加**virtual**关键字，C++编译器能自动分辨这种二义性。

✓派生类的对象在构造时对虚基类的子对象只构造一次——有效解决二义性问题。




```
class Bed :virtual public Furniture    {…};
class Sofa :virtual public Furniture   {…};
class SleeperSofa :public Bed , public Sofa
   {…};
```

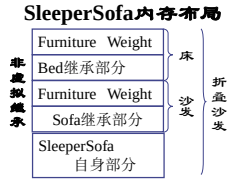

4. 多继承

虚继承（虚基类）

```
#include<iostream.h>
class Furniture{
public:
  Furniture(){}
  void SetWeight(int i){ weight =i; }
  int GetWeight(){ return weight; }
protected:
  int weight;
};
class Bed :virtual public Furniture{
public:
  Bed(){}
  void Sleep(){ cout <<"Sleeping...\n"; }
};
class Sofa :virtual public Furniture{
public:
  Sofa(){}
  void WatchTV(){ cout <<"Watching TV.\n"; }
};
class SleeperSofa :public Bed, public Sofa{
public:
  SleeperSofa() :Sofa(), Bed(){}
  void FoldOut(){ cout <<"Fold out the sofa.\n"; }
};

void main()
{  SleeperSofa ss;
  ss.SetWeight(20);
  cout <<ss.GetWeight() <<endl;
}
```

SleeperSofa内存布局



4. 多继承

多继承的构造函数与析构函数

多继承构造函数的定义形式与单继承构造函数的定义形式相似，只是n个基类的构造函数之间用“，”分隔。多重继承构造函数定义的一般形式如下：

派生类名::派生类名(参数总表):基类名(参数表1),…,基类名(参数表n),子对象名1(对象参数表1),…,子对象名m(对象参数表m)

{

派生类新增加成员的初始化；

}

4. 多继承

多继承的构造函数与析构函数

构造/析构函数执行顺序

当不存在虚基类时，多重继承的构造函数的执行顺序与单继承构造函数的执行顺序相同：

先执行基类的构造函数
再执行对象成员的构造函数
最后执行派生类构造函数。

在多个基类之间，则严格按照派生类声明时从左到右的顺序来排列先后。而析构函数的执行顺序则刚好与构造函数的执行顺序相反。

```
class Base1
   {…};
class Base2
   {…};
class Derived: Base1,Base2
   {Base1 X;
      …};
int main( )
   { Derived obj;
      ….
   }
```

Base1→Base2 → Base1 → Derived

4. 多继承

多继承的构造函数与析构函数

构造/析构函数执行顺序

当存在虚基类时：

(1) **虚基类的构造函数在非虚基类之前调用。**

```
class Base1
   {…};
class Base2
   {…};
class Derived: Base1,virtual Base2
   {…};
int main( )
   { Derived obj;
      ….
   }
```

Base2→Base1 → Derived

4. 多继承

多继承的构造函数与析构函数

构造/析构函数执行顺序

当存在虚基类时：

(1) 虚基类的构造函数在非虚基类之前调用。

(2) 若同一层次中包含多个虚基类，这些虚基类的构造函数按对它们说明的先后次序调用。

```
class Base1
  {…};
class Base2
  {…};
class Derived: virtual Base1, virtual Base2
  {…};
int main( )
  { Derived obj;
    ….
  }
```

Base1→Base2 → Derived

43

4. 多继承

多继承的构造函数与析构函数

构造/析构函数执行顺序

当存在虚基类时：

(1) 虚基类的构造函数在非虚基类之前调用。

(2) 若同一层次中包含多个虚基类，这些虚基类的构造函数按对它们说明的先后次序调用。

(3) 若虚基类由非虚基类派生而来，则仍然先调用基类构造函数，再调用派生类的构造函数。

(4) 析构函数的调用顺序与构造函数调用顺序相反。

```
class GBase
   {…};
class FBase:GBase
{…};
class Derived: virtual public FBase
{…};

int main()
{  Derived obj;
   return 0;
}
```

GBase→FBase → Derived

44

END

50